

Assignment 01

Clustering and Classification — K-Means, DBSCAN, and KNN

Student: Abu Bakar, Student **Reg No:** NUM-BSCS-2022-41, **Date:** November 27, 2025

1. Introduction

This assignment explores the application and comparative performance of two unsupervised clustering algorithms, **K-Means** and **DBSCAN**, and one supervised classification algorithm, **K-Nearest Neighbors (KNN)**. The analysis is performed on two synthetic 2D datasets: a clean dataset with four distinct, convex clusters, and a complex dataset that includes noise and a non-convex cluster. The goal is to understand how these algorithms handle different data distributions and the presence of noise.

The entire analysis was conducted using Python with the scikit-learn, pandas, matplotlib, and seaborn libraries.

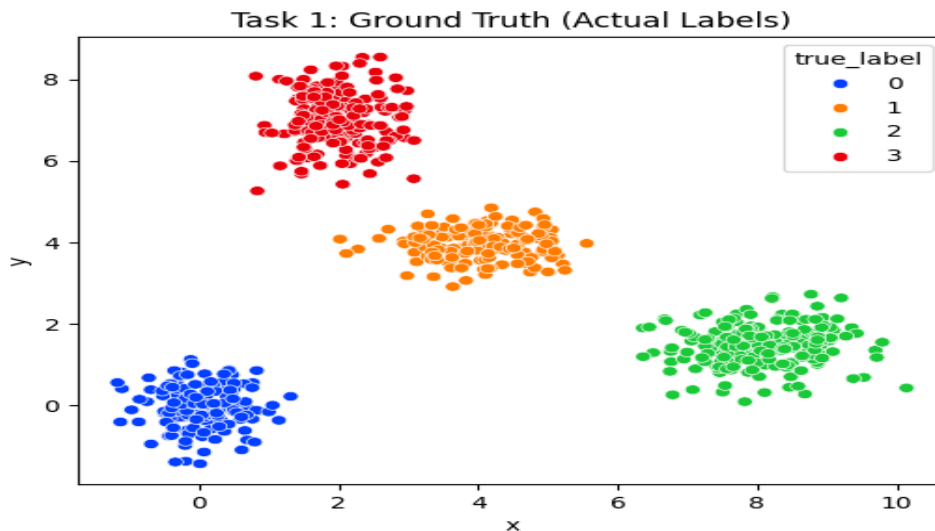
2. Dataset Analysis (Task 1)

The assignment uses two datasets. Both datasets contain features x and y, and a true_label column for ground truth comparison.

2.1. Dataset 1: Clean Data (No Noise)

This dataset features four clearly separated, convex clusters.

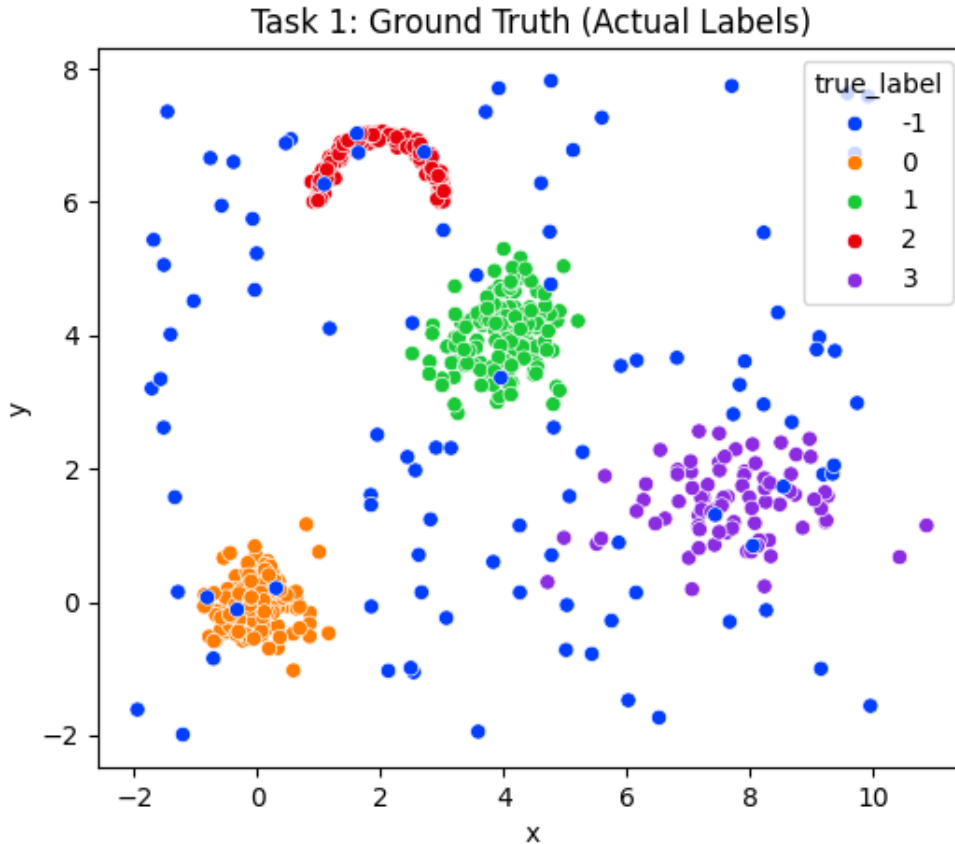
Figure 1: Ground Truth (Actual Labels) - Clean Data



2.2. Dataset 2: Complex Data (With Noise)

This dataset is more challenging. It contains four main clusters, including one non-convex (C-shaped) cluster, and a significant amount of scattered noise points.

Figure 2: Ground Truth (Actual Labels) - Complex Data with Noise



3. K-Means Clustering (Task 2)

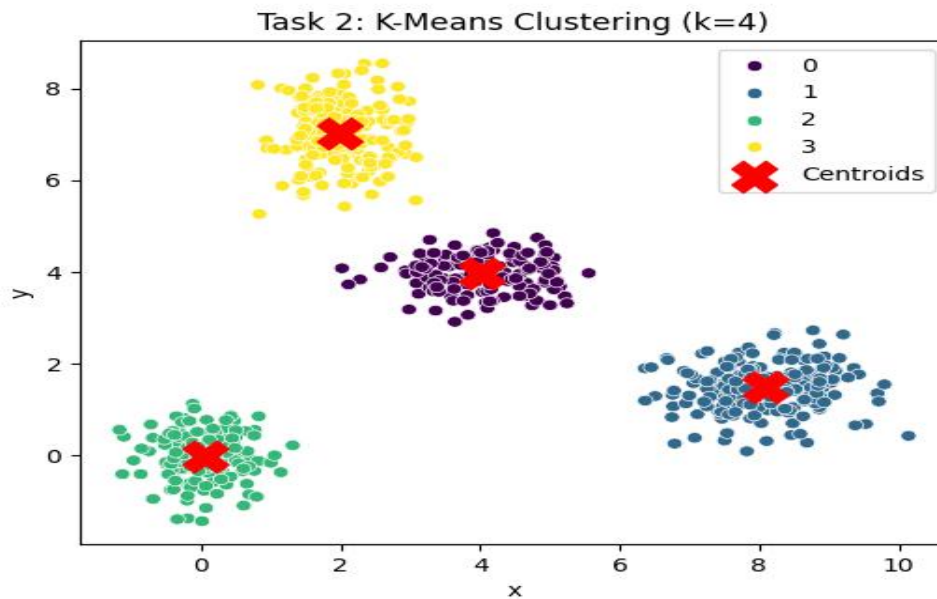
K-Means is a simple, iterative, and partition-based clustering algorithm. It aims to partition N observations into K clusters, where each observation belongs to the cluster with the nearest mean (centroid).

Configuration: * **Number of Clusters (k):** 4 (as specified by the task and suggested by the ground truth).

3.1. K-Means on Clean Data

K-Means performs exceptionally well on the clean dataset because the clusters are well-separated, convex, and of similar density. The algorithm successfully identifies the four natural groupings.

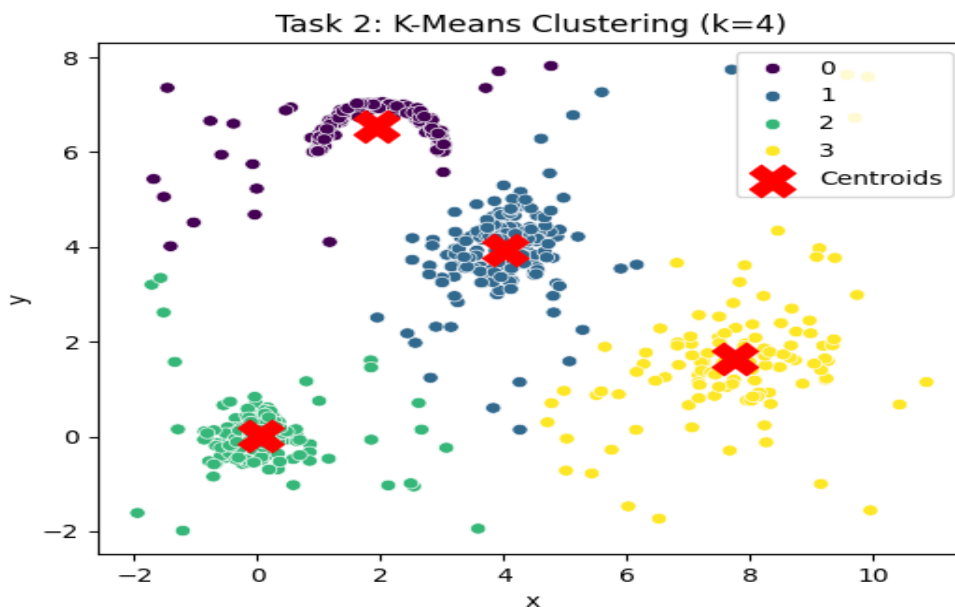
Figure 3: K-Means Clustering ($k = 4$) - Clean Data



3.2. K-Means on Complex Data (With Noise)

The performance degrades significantly on the complex dataset. 1. **Noise Sensitivity:** K-Means is highly sensitive to noise. The scattered noise points pull the cluster centroids towards them, distorting the shape and position of the clusters. 2. **Convexity Limitation:** The algorithm struggles with the non-convex (C-shaped) cluster, as it forces a convex boundary, incorrectly splitting the C-shape and merging parts of it with the noise points.

Figure 4: K-Means Clustering ($k = 4$) - Complex Data with Noise



4. DBSCAN Clustering (Task 3)

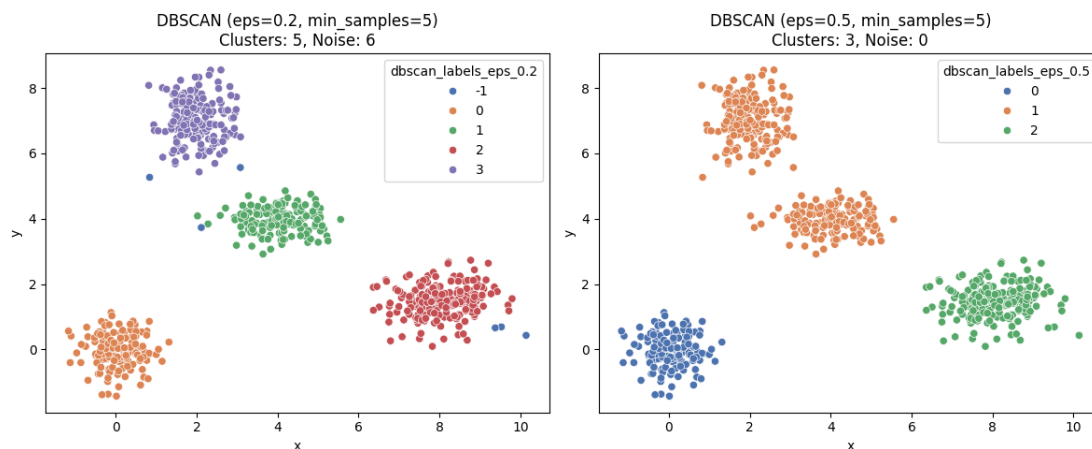
DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a density-based clustering algorithm. It groups together points that are closely packed (having many neighbors) and marks points that lie alone in low-density regions as outliers (noise).

Configuration: * **Minimum Samples (min_samples):** 5 (Constant for both runs). * **Epsilon (eps):** Two values were tested: 0.2 and 0.5.

4.1. DBSCAN on Clean Data

ϵ	Clusters Found	Noise Points	Observation
0.2	5	6	A smaller ϵ leads to over-segmentation, splitting one of the true clusters into two, but successfully ignores a few boundary points as noise.
0.5	3	0	A larger ϵ causes two adjacent clusters to be merged into a single cluster, but correctly identifies the three remaining clusters with no noise.

Figure 5: DBSCAN Clustering - Clean Data

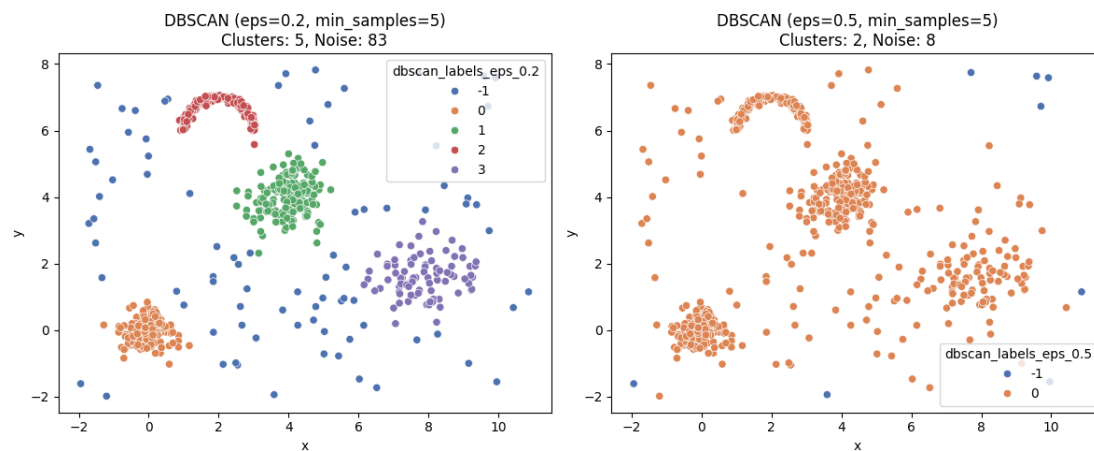


4.2. DBSCAN on Complex Data (With Noise)

DBSCAN is particularly effective here due to its ability to handle non-convex shapes and explicitly identify noise.

ϵ	Clusters Found	Noise Points	Observation
0.2	5	83	This value successfully identifies the non-convex (C-shaped) cluster and the other dense clusters. Crucially, it correctly labels the majority of the scattered points as noise (83 points).
0.5	2	8	This value is too large, causing multiple dense clusters to be merged into two large clusters. The noise count is lower, indicating that many noise points were absorbed into the merged clusters.

Figure 6: DBSCAN Clustering - Complex Data with Noise



5. K-Nearest Neighbors (KNN) Classification (Task 4)

K-Nearest Neighbors (KNN) is a non-parametric, supervised learning method used for classification. A data point is classified by a majority vote of its k nearest neighbors, with the point being assigned to the class most common among its k nearest neighbors.

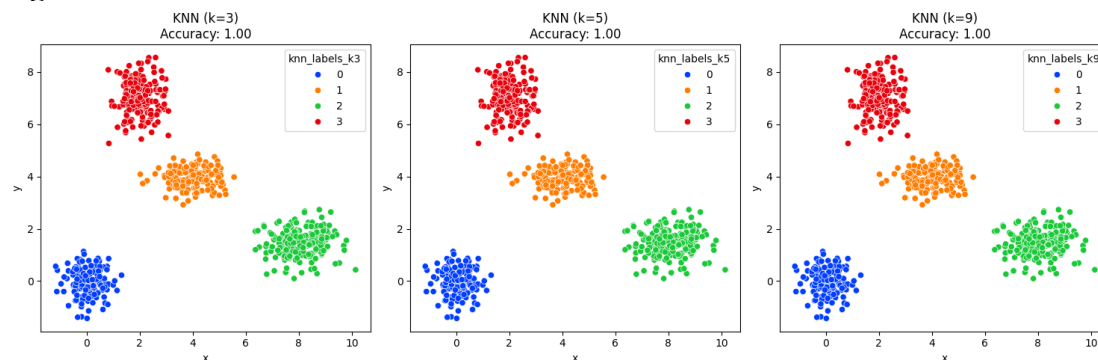
Configuration: * k values tested: 3, 5, and 9. * **Data Split:** 80% for training, 20% for testing.

5.1. KNN on Clean Data

The clean, well-separated data results in perfect classification accuracy across all tested k values.

k	Accuracy
3	1.00
5	1.00
9	1.00

Figure 7: KNN Classification - Clean Data



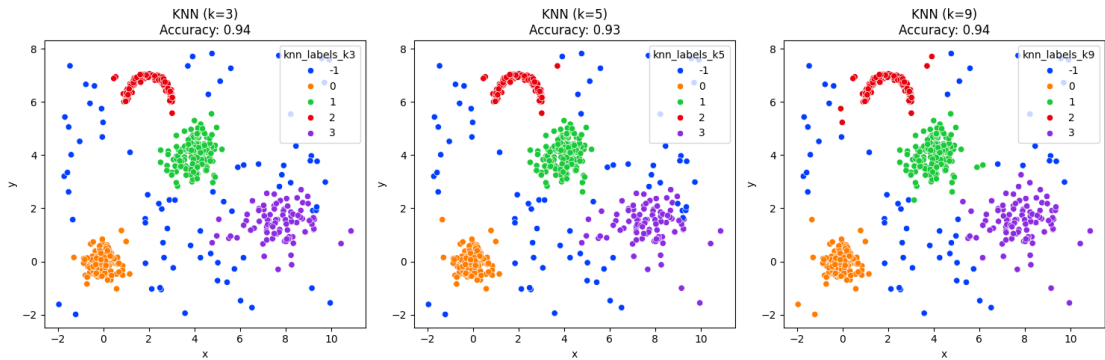
5.2. KNN on Complex Data (With Noise)

The presence of noise and the non-convex shape slightly reduce the accuracy, as some noise points may influence the classification of nearby test points.

<i>k</i>	Accuracy
3	0.94
5	0.93
9	0.94

The accuracy remains high (around 94%), demonstrating KNN’s robustness, especially when the underlying classes are still relatively distinct. The slight drop in accuracy for $k = 5$ compared to $k = 3$ and $k = 9$ suggests a minor sensitivity to the local distribution of the noise points.

Figure 8: KNN Classification - Complex Data with Noise



6. Comparative Discussion

Feature	K-Means	DBSCAN	KNN
Type	Unsupervised (Partitioning)	Unsupervised (Density-Based)	Supervised (Classification)
Input Parameter	Number of clusters (k)	ϵ (radius) and min_samples	Number of neighbors (k)
Handling Noise	Poor (Noise distorts centroids)	Excellent (Explicitly labels noise as -1)	Good (Robust, but noise can affect local decision boundaries)
Handling Shape	Poor (Assumes convex/spherical)	Excellent (Handles non-convex shapes)	Excellent (Decision boundary is highly flexible)
Clean Data Performance	Excellent (Accuracy 1.00)	Good (Sensitive to ϵ choice)	Excellent (Accuracy 1.00)
Complex Data Performance	Poor (Centroids pulled by noise)	Excellent (Correctly identifies noise and non-convex cluster with optimal ϵ)	Very Good (Accuracy ≈ 0.94)

Summary of Findings:

1. **K-Means** is the simplest and fastest method, but it is only suitable for data with well-defined, spherical clusters and no significant noise. Its performance on the complex dataset was the worst among the three.
 2. **DBSCAN** proved to be the most powerful clustering algorithm for the complex dataset. By correctly setting the ϵ parameter (e.g., $\epsilon = 0.2$), it successfully identified the non-convex cluster and isolated the scattered noise points, which K-Means failed to do. However, its performance is highly dependent on the correct selection of its two parameters.
 3. **KNN** demonstrated strong performance as a classifier on both datasets. Even with the presence of noise, the classification accuracy remained high, highlighting its flexibility in creating complex decision boundaries based on local data density.
-

7. Source Code

The analysis was performed using two Python scripts, one for each dataset.

7.1. Code for Clean Data (Kmeans_DBSCAN_KNN_code_no_noise.py)

```
python import pandas as pd import matplotlib.pyplot as plt import seaborn as sns from
sklearn.cluster import KMeans, DBSCAN from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split from sklearn.metrics import
accuracy_score, confusion_matrix from sklearn.preprocessing import StandardScaler
```

1. Load the Dataset

```
filename = 'Kmeans_DBScan_KNN_convex_no_noise.csv'
```

```
try: df = pd.read_csv(filename) print("Dataset loaded successfully.") print(df.head()) except
FileNotFoundError: print(f'Error: {filename} not found. Please ensure the file is in the same
directory.') exit()
```

2. Preprocessing

```
X = df[['x', 'y']] # Scaling is crucial for distance-based algorithms like K-Means, DBSCAN, and
KNN scaler = StandardScaler() X_scaled = scaler.fit_transform(X)
```

— Task 1: Load and Analyze Dataset —

```
print("— Task 1: Dataset Visualization —") plt.figure(figsize=(6, 5)) sns.scatterplot(data=df,
x='x', y='y', hue='true_label', palette='bright', legend='full') plt.title('Task 1: Ground Truth
(Actual Labels)') plt.savefig('Task1_GroundTruth.png') # plt.show()
```

— Task 2: K-Means Clustering —

```
print("— Task 2: K-Means Clustering (k=4) —") k_means_k = 4 # Initialize and fit K-Means on
the scaled data kmeans = KMeans(n_clusters=k_means_k, random_state=42, n_init=10)
df['kmeans_labels'] = kmeans.fit_predict(X_scaled)
```

Inverse transform centroids to plot on the original scale

```
centroids_scaled = kmeans.cluster_centers_
centroids = scaler.inverse_transform(centroids_scaled)

plt.figure(figsize=(6, 5)) sns.scatterplot(data=df, x='x', y='y', hue='kmeans_labels',
palette='viridis', legend='full') plt.scatter(centroids[:, 0], centroids[:, 1], s=300, c='red',
marker='X', label='Centroids') plt.title(f'Task 2: K-Means Clustering (k={k_means_k})')
plt.legend() plt.savefig('Task2_KMeans.png') # plt.show()
```

— Task 3: DBSCAN Clustering —

```
print("— Task 3: DBSCAN Clustering —") eps_values = [0.2, 0.5] min_samples = 5

plt.figure(figsize=(12, 5))

for i, eps in enumerate(eps_values): dbscan = DBSCAN(eps=eps, min_samples=min_samples)
col_name = f'dbscan_labels_eps_{eps}' df[col_name] = dbscan.fit_predict(X_scaled)

# Calculate number of clusters (excluding noise label -1) and noise points
n_clusters_db = len(set(df[col_name])) - (1 if -1 in df[col_name].values else 0)
n_noise_db = list(df[col_name]).count(-1)
print(f'DBSCAN (eps={eps}): Clusters={n_clusters_db}, Noise Points={n_noise_db}')

plt.subplot(1, 2, i+1)
sns.scatterplot(data=df, x='x', y='y', hue=col_name, palette='deep', legend='full')
plt.title(f'DBSCAN (eps={eps}, min_samples={min_samples})\nClusters: {n_clusters_db}, Noise:
{n_noise_db}')

plt.tight_layout() plt.savefig('Task3_DBSCAN.png') # plt.show()
```

— Task 4: KNN Classification —

```
print("— Task 4: KNN Classification —") y = df['true_label'] # Split data for training and
testing X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2,
random_state=42)

k_values = [3, 5, 9] plt.figure(figsize=(15, 5))

for i, k in enumerate(k_values): knn = KNeighborsClassifier(n_neighbors=k) knn.fit(X_train,
y_train)
```



```

y_pred = knn.predict(X_test)
acc = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

print(f"\nKNN (k={k}):")
print(f"Accuracy: {acc:.4f}")
print("Confusion Matrix:\n{cm}")

# Predict on the whole dataset for visualization
df[f'knn_labels_k{k}'] = knn.predict(X_scaled)

plt.subplot(1, 3, i+1)
sns.scatterplot(data=df, x='x', y='y', hue=f'knn_labels_k{k}', palette='bright', legend='full')
plt.title(f'KNN (k={k})\nAccuracy: {acc:.2f}')

plt.tight_layout() plt.savefig('Task4_KNN.png') # plt.show()

print("— Execution Completed —")

```

7.2. Code for Complex Data with Noise (Kmeans_DBSCAN_KNN_code_noise.py)

```

python import pandas as pd import matplotlib.pyplot as plt import seaborn as sns from
sklearn.cluster import KMeans, DBSCAN from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split from sklearn.metrics import
accuracy_score, confusion_matrix from sklearn.preprocessing import StandardScaler

```

1. Load the Dataset

```

filename = 'Kmeans_DBScan_KNN_complex_noise.csv'

try: df = pd.read_csv(filename) print("Dataset loaded successfully.") print(df.head()) except
FileNotFoundError: print(f'Error: {filename} not found. Please ensure the file is in the same
directory.') exit()

```

2. Preprocessing

```

X = df[['x', 'y']] # Scaling is crucial for distance-based algorithms like K-Means, DBSCAN, and
KNN scaler = StandardScaler() X_scaled = scaler.fit_transform(X)

```

— Task 1: Load and Analyze Dataset —

```

print("— Task 1: Dataset Visualization —") plt.figure(figsize=(6, 5)) sns.scatterplot(data=df,
x='x', y='y', hue='true_label', palette='bright', legend='full') plt.title('Task 1: Ground Truth
(Actual Labels)') plt.savefig('Task1_GroundTruth_noise.png') # plt.show()

```

— Task 2: K-Means Clustering —

```
print("— Task 2: K-Means Clustering (k=4) —") k_means_k = 4 # Initialize and fit K-Means on
the scaled data kmeans = KMeans(n_clusters=k_means_k, random_state=42, n_init=10)
df['kmeans_labels'] = kmeans.fit_predict(X_scaled)
```

Inverse transform centroids to plot on the original scale

```
centroids_scaled = kmeans.cluster_centers_
centroids = scaler.inverse_transform(centroids_scaled)

plt.figure(figsize=(6, 5)) sns.scatterplot(data=df, x='x', y='y', hue='kmeans_labels',
palette='viridis', legend='full') plt.scatter(centroids[:, 0], centroids[:, 1], s=300, c='red',
marker='X', label='Centroids') plt.title(f'Task 2: K-Means Clustering (k={k_means_k})')
plt.legend() plt.savefig('Task2_KMeans_noise.png') # plt.show()
```

— Task 3: DBSCAN Clustering —

```
print("— Task 3: DBSCAN Clustering —") eps_values = [0.2, 0.5] min_samples = 5

plt.figure(figsize=(12, 5))

for i, eps in enumerate(eps_values): dbscan = DBSCAN(eps=eps, min_samples=min_samples)
col_name = f'dbscan_labels_eps_{eps}' df[col_name] = dbscan.fit_predict(X_scaled)

# Calculate number of clusters (excluding noise label -1) and noise points
n_clusters_db = len(set(df[col_name])) - (1 if -1 in df[col_name].values else 0)
n_noise_db = list(df[col_name]).count(-1)
print(f'DBSCAN (eps={eps}): Clusters={n_clusters_db}, Noise Points={n_noise_db}')

plt.subplot(1, 2, i+1)
sns.scatterplot(data=df, x='x', y='y', hue=col_name, palette='deep', legend='full')
plt.title(f'DBSCAN (eps={eps}, min_samples={min_samples})\nClusters: {n_clusters_db}, Noise:
{n_noise_db}')

plt.tight_layout() plt.savefig('Task3_DBSCAN_noise.png') # plt.show()
```

— Task 4: KNN Classification —

```
print("— Task 4: KNN Classification —") y = df['true_label'] # Split data for training and
testing X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2,
random_state=42)

k_values = [3, 5, 9] plt.figure(figsize=(15, 5))

for i, k in enumerate(k_values): knn = KNeighborsClassifier(n_neighbors=k) knn.fit(X_train,
y_train)
```

```

y_pred = knn.predict(X_test)
acc = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

print(f"\nKNN (k={k}):")
print(f"Accuracy: {acc:.4f}")
print("Confusion Matrix:\n{cm}")

# Predict on the whole dataset for visualization
df[f'knn_labels_k{k}'] = knn.predict(X_scaled)

plt.subplot(1, 3, i+1)
sns.scatterplot(data=df, x='x', y='y', hue=f'knn_labels_k{k}', palette='bright', legend='full')
plt.title(f'KNN (k={k})\nAccuracy: {acc:.2f}')

plt.tight_layout() plt.savefig('Task4_KNN_noise.png') # plt.show()

print("— Execution Completed —")

```

8. Chat Session Transcript

This section contains the complete chat session with the generative tool (Gemini) used to complete the assignment.

User Prompt 1: > I have uploaded a CSV file which consist the dataset without noise

Analyze this dataset csv file

Apply the Clustering Algorithms on this dataset (Kmean, DBSCAN)

Write code in python and write simple code

User Prompt 2: > add KNN in this code also on the same dataset

Write code in simple python after these 2 Algorithms

User Prompt 3: > Gives the Assignment Guidelines and prompt (Antigravity, Google Agentic IDE using Gemini 3.0)

Analyze these Assignment Guidelines and Change the code from both files according to the guidelines and save the plots don't leave any step and write simple python code